

Task 1 — Web App QA & Debug Report

Automation & QA Developer — Take-Home Assessment · Candidate: **Firoz Ahmad**

Application under test	Conduit (RealWorld reference app) — Angular frontend
Tested URL	<code>https://demo.realworld.show</code> (API: <code>https://api.realworld.show</code>)
Why this URL	The assignment's <code>https://demo.realworld.io</code> returns HTTP 404 (verified live). I used the maintained demo linked from the RealWorld repo README, per the brief. The dead <code>.io</code> link is <i>not</i> reported as an app bug.
Backend note	The landing page states the demo backend "enforces session isolation." Relevant to one anomaly I investigated (see Evidence Gaps).
Environment	Windows 11 · Chromium 153 (Playwright-driven) · desktop viewport
Date / time of test	2026-09-29, ~16:39–16:50 UTC (machine clock reports UTC)
Method	Live black-box testing of the main user flows (sign-up, login, create, edit, delete, comment, logout, protected route) plus validation, accessibility (DOM audit), content-sanitization, and network-layer inspection of API requests/responses. Screenshots and captured HTTP status/payloads are the evidence. Secrets/tokens redacted.
Test account	Disposable, no sensitive data. Password/token values redacted throughout.

Summary

I exercised every core flow. **Six distinct, reproducible issues** are reported below (1 High, 4 Medium, 1 Low), each backed by captured evidence. Several behaviors I checked turned out to be **correct** and are listed under "Correct behaviors" so bugs are not invented to reach a quota. One surprising observation (an unexpected username) was investigated and traced to the *shared demo backend*, not the app — documented honestly under Evidence Gaps rather than counted as a bug.

1 High 4 Medium 1 Low

Bug Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Sign-up accepts a malformed email and a 1-character password	Go to <code>/register</code> . Enter Username <code>fzqatest</code> , Email <code>notanemail</code> (no "@"), Password <code>1</code> . Click Sign up .	Expected: invalid email + trivial password are rejected; no account created. Actual: button enables; <code>POST /api/users</code> → 201 Created ; user is logged in; email <code>notanemail</code> is stored.	High	No email-format or password-strength validation on <i>either</i> client (form only checks non-empty) <i>or</i> server (API persists it, 201). <i>Confirmed at network layer.</i>

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
2	Every route uses the same static page title “Conduit”	Visit <code>/</code> , <code>/login</code> , <code>/register</code> , <code>/editor</code> , <code>/settings</code> , <code>/article/<slug></code> . Observe the browser tab / <code>document.title</code> .	Expected: title changes per route (e.g. “Sign in — Conduit”). Actual: <code>document.title</code> is “Conduit” everywhere; only the on-page <code><h1></code> changes.	Medium	SPA router never sets a per-route title (no Angular <code>Title</code> service / route title resolver). <i>Confirmed.</i>
3	Form fields have no real labels; email is <code>type="text"</code>; no autocomplete	On <code>/register</code> or <code>/login</code> , inspect the inputs (DOM audit).	Expected: programmatic <code><label></code> / <code>aria-label</code> per field; email <code>type="email"</code> ; autocomplete hints. Actual: 0 <code><label></code> elements (placeholder-only), email <code>type="text"</code> , no <code>autocomplete</code> anywhere.	Medium	Template relies on placeholders instead of labels; input type/attributes not set. <i>Confirmed structurally.</i>
4	Edit-article form does not load the article’s existing tags	Create an article with tag <code>qatest</code> . Open Edit Article . Look at the Tags field.	Expected: existing tags pre-populate and are removable. Actual: title/description/body pre-fill, but Tags field is empty (0 pills) though the article is tagged <code>qatest</code> .	Medium	Edit component doesn’t map <code>article.tagList</code> into the tags form control on load. Backend keeps the tag, so no data loss observed — but tags are hidden/unmanageable in the UI. <i>Confirmed on client.</i>
5	“Delete Article” is immediate & irreversible — no confirmation	Open your article. Click Delete Article .	Expected: a confirm prompt before permanent deletion. Actual: deletes instantly (<code>DELETE</code> → 204), redirects home; no dialog, no undo.	Medium	Delete handler calls the API directly on click with no <code>confirm()</code> / modal guard. <i>Confirmed.</i>
6	Article body renders raw HTML passthrough (active script XSS is mitigated)	Publish an article whose body contains <code></code> , <code><i></code> , <code></code> , <code><script></code> . Inspect rendered output.	Expected: raw HTML is escaped or consistently sanitized to a safe subset. Actual: <code><script></code> stripped and <code>onerror</code> stripped (no code ran — good), but raw <code></code> / <code><i></code> / <code></code> render as live HTML.	Low	Markdown pipeline lets a subset of raw HTML through the sanitizer (scripts/handlers removed, formatting tags/img kept). <i>Confirmed.</i>

Detailed Findings — Plain-English Impact & Evidence

#1 · Sign-up accepts a malformed email and a 1-character password High

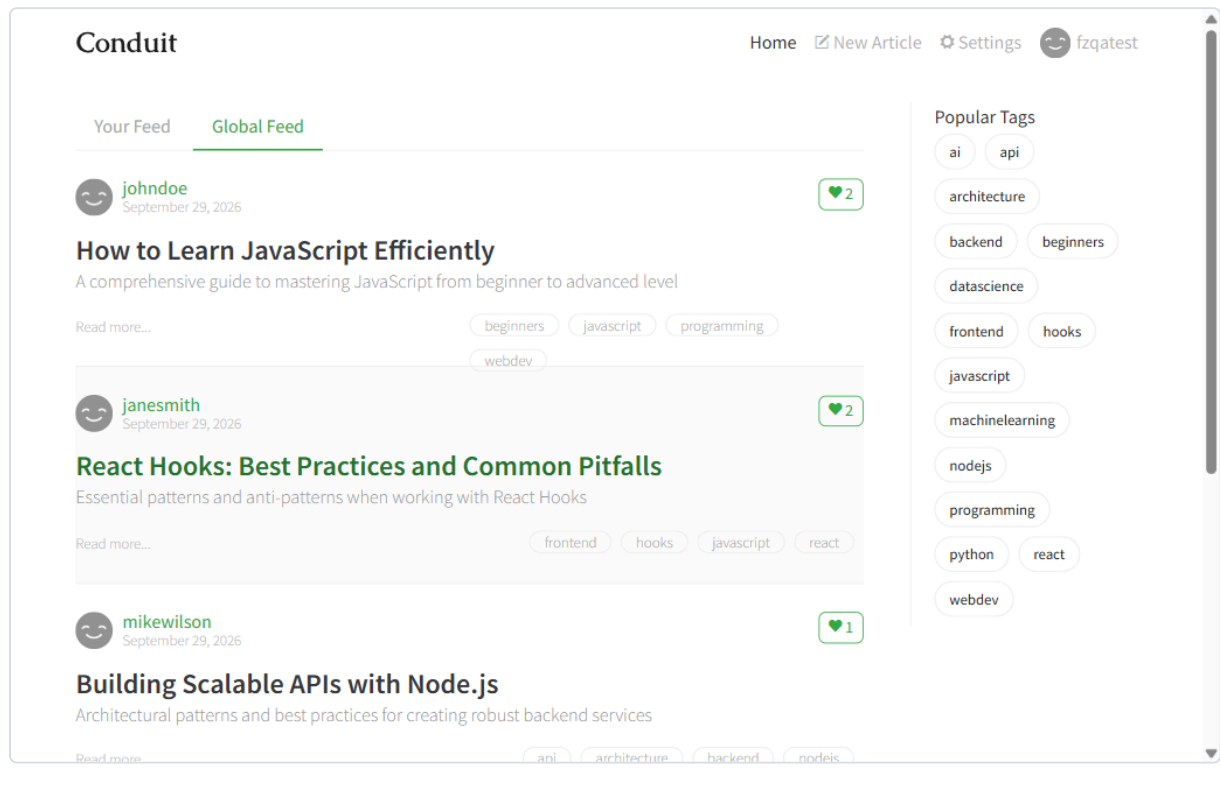
Impact: Users can register with addresses that can never receive mail, so email verification, password reset and all notifications silently fail for them. A 1-character password makes accounts trivially guessable — a real account-takeover and support-cost risk in production.

Evidence (confirmed):

Request body: `{"user":{"email":"notanemail","password":"1","username":"fzqatest"}}`

Response: **HTTP 201 Created** — account created, auth token returned (*token redacted*).

Settings page later displays the stored email as `notanemail`. The “Sign up” button became enabled with these invalid values.



#2 · Static page title on every route Medium

Impact: WCAG 2.2 **2.4.2 (Page Titled)** failure. Screen-reader users hear “Conduit” on every page and can’t tell them apart; users with many tabs can’t distinguish them; bookmarks and search-engine results all share one title, hurting orientation and SEO.

Evidence (confirmed): `document.title === "Conduit"` on `/register` while `<h1> === "Sign up"`; the reported “Page Title: Conduit” was identical across `/`, `/login`, `/register`, `/editor`, `/settings` and `/article/<slug>` during navigation.

#3 · Missing labels, `type="text"` email, no autocomplete Medium

Impact: WCAG **1.3.1 / 3.3.2 / 4.1.2** (and 1.3.5). Because fields use placeholder text as their only label, the label vanishes the moment the user types, and screen readers may announce no accessible name.

`type="text"` for email means no browser-level validation and the wrong (non-email) keyboard on mobile. Absent `autocomplete` hurts password-manager and autofill reliability.

Evidence (confirmed): DOM audit → `labelElementCount = 0` ; Email input `type = "text"` ; every input `autocomplete = null` .

Conduit

HomeSign inSign up

Sign up

Have an account?

Username

Email

Password

Sign up

Conduit

© 2026. An interactive learning project from RealWorld OSS Project. Code licensed under MIT.

#4 · Edit form drops the visible tags Medium

Impact: When editing, users can't see or remove the tags already on their article, which is confusing and leads to inconsistent tagging. The data isn't lost on the backend in this build, but the UI misrepresents the article's real state.

Evidence (confirmed): On `/editor/<slug>`, `tagsFieldValue = ""` and 0 tag pills, while the published article shows tag `qatest`. After saving a title change without re-typing tags, `qatest` was still present on the article.

#5 · Delete with no confirmation Medium

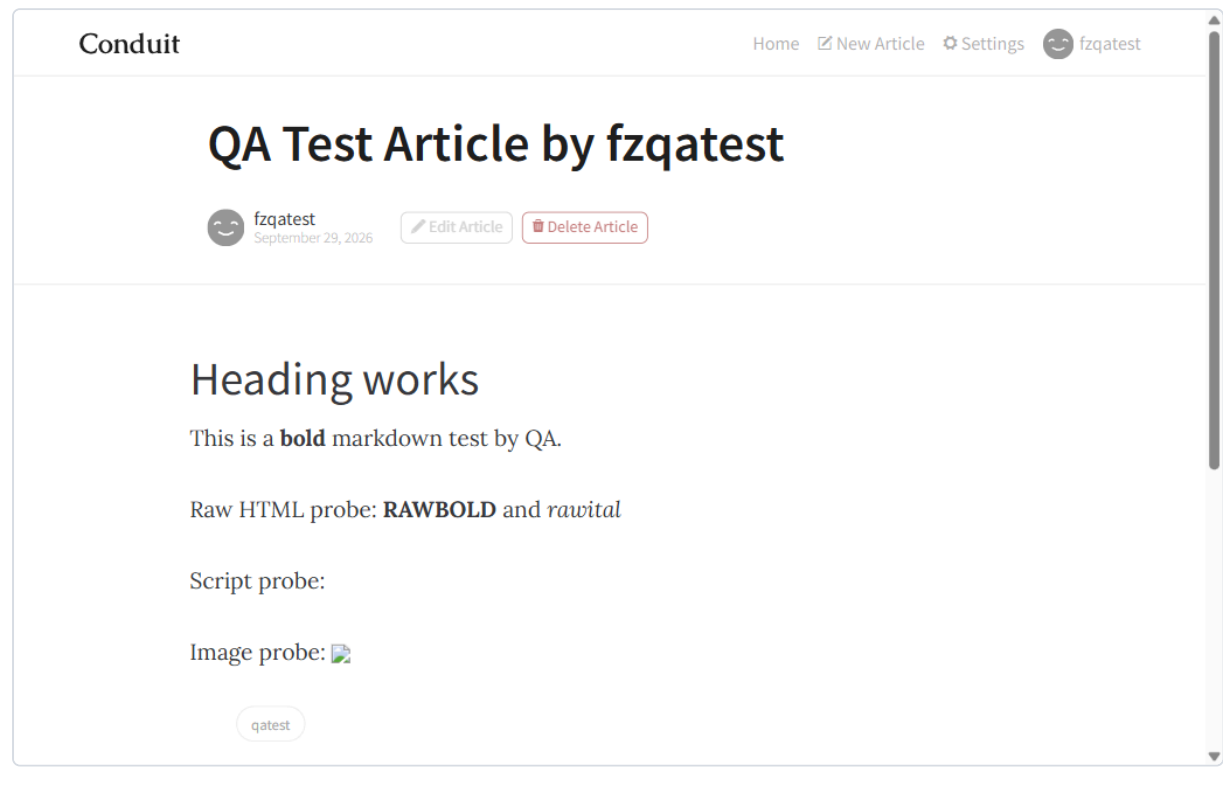
Impact: A single accidental click permanently destroys an article with no "are you sure?" and no undo — a genuine data-loss hazard, especially on touch devices where mis-taps are common.

Evidence (confirmed): Clicking **Delete Article** fired no dialog; `DELETE /api/articles/<slug>` → **204 No Content**; app immediately redirected to `/` and the article was gone.

#6 · Raw HTML passthrough in article body Low

Impact: This build *does* neutralize the dangerous vectors (script tags and event handlers are removed — I verified no code executed), so this is **not** an active script-XSS. The residual risk is that arbitrary formatting HTML and bare `<img>` tags render, enabling layout/UI spoofing inside content, off-site image hotlinking / tracking pixels, and a fragile sanitizer that could regress into something exploitable if its config changes.

Evidence (confirmed): After publishing a probe article, `window.__xss_script` and `window.__xss_img` were both `false` (nothing executed). Rendered DOM contained `<b>RAWBOLD</b>` and ``; the `<script>` body and the `onerror` attribute were stripped.



Root-Cause Analysis (Finding #1 — weak sign-up validation)

The sign-up form is an Angular reactive form whose only validators appear to be `required` on the three controls, so the “Sign up” button enables as soon as each field is non-empty. There is no `validators.email` /pattern check on the email control and no minimum-length or complexity rule on the password control. On submit, the frontend passes the raw values straight to `POST /api/users`. The demo API then accepts the payload and returns **201 Created**, persisting the email `notanemail` verbatim — which proves the gap exists on *both* tiers, not just the UI. This is a **confirmed** cause at the network layer (I captured the 201 response and later saw the stored email in Settings), not a hypothesis. The fix is defense-in-depth: on the client add `validators.email` (or a stricter RFC-lite pattern) and a password policy (e.g. min length 8 with basic complexity) for instant feedback; on the server enforce the same and return **422** with field errors for invalid email or weak passwords. The app already renders server-side 422 errors through its `.error-messages` list (as seen on empty-article and empty-comment submissions), so surfacing these new errors is low-effort. Email addresses should additionally pass a verification step before an account is treated as active, so unreachable values like `notanemail` can’t hold an account. Until then, password reset, email verification and notifications are guaranteed to fail for anyone who signs up with an invalid address.

Correct Behaviors Verified (not bugs)

- ✓ Publishing an **empty article** is rejected and the UI shows "title/description/body can't be blank" (server 422 surfaced correctly).
- ✓ Posting an **empty comment** shows "body can't be blank".
- ✓ **Failed login** (correct email, wrong password) returns 401 and shows "credentials invalid" — clear, no crash.
- ✓ **Protected route** /settings redirects to /login when logged out (route guard works).
- ✓ Password field is correctly type="password" (masked); login and logout work.

Evidence Gaps & Honest Notes

Investigated anomaly — NOT counted as a bug. After registering as `fzqatest` (email `notanemail`) and later logging back in, the authenticated username showed as `"a"` instead of `fzqatest`. Rather than report this as an app defect, I disambiguated it: a fresh registration with a *unique* username/email echoed the username back correctly (`usernameMatches: true`, 201). The likely cause is that the **shared public demo backend** keys accounts by email and the very common test string `notanemail` collides with other testers' data (someone had renamed that account to `"a"`). This is an artifact of my test-data choice on a shared backend, not a reproducible application bug — so I excluded it.

- **Not tested:** real email deliverability; load/performance and network-throttling (the brief prohibits load-testing the public demo); full automated axe-core sweep (I used targeted DOM/ARIA checks + manual reasoning, which cover only part of WCAG); cross-browser (Chromium only); mobile-viewport layout.
- **Confirmed vs hypothesis:** every row above is confirmed with a captured HTTP status/payload, DOM value, or absence-of-dialog observation. Where I inferred an implementation cause (e.g., "no Title service"), it is labelled as a suspected cause.